

ほぼAIだよりで作成したシステムの課題・反省点

何があったのか

GCP,Terraform,Fast API,FreeSwitch(PBX),Kamailio(SIP Proxy)を使ったシステムを開発。

実装のすべてをAI(Claude Code)に任せて進めた。

暦日75日・稼働52日でmainに498コミット。

パット見数字だけ見ると順調そう(?)

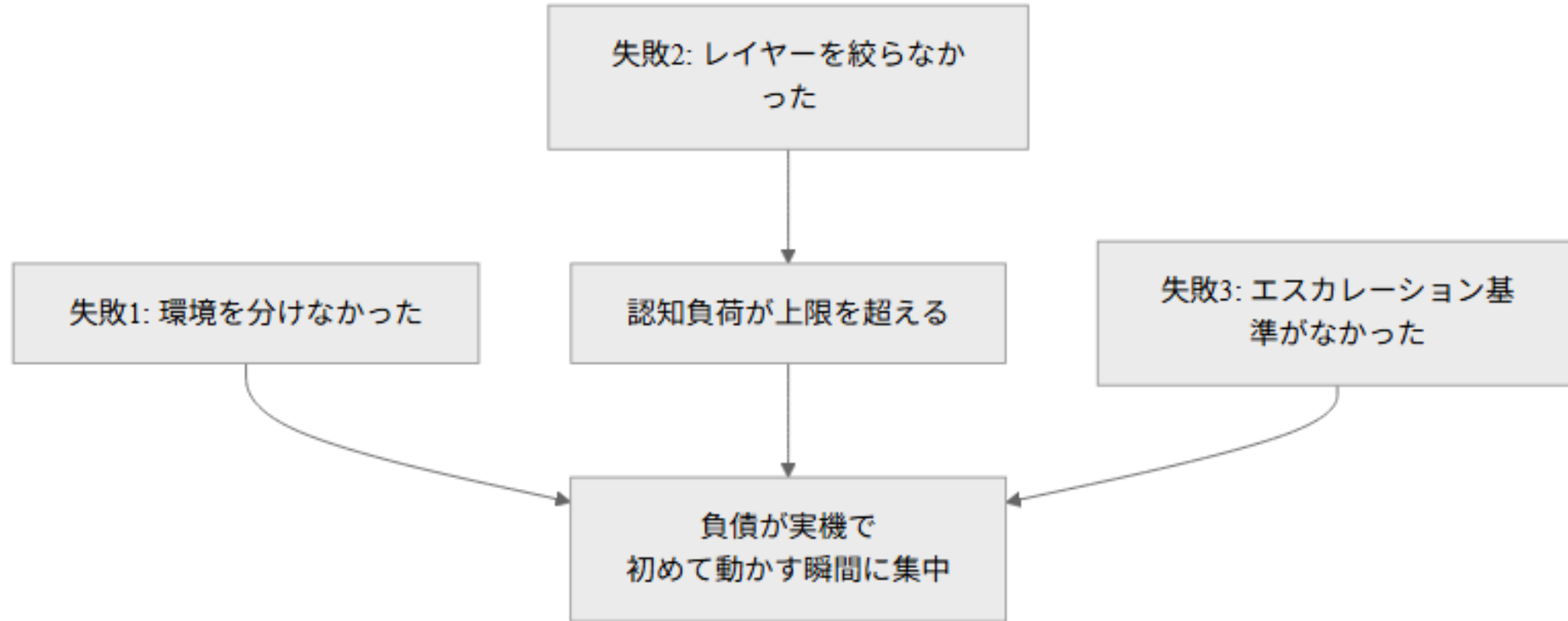
結論

システムそのものは、なんだかんだありつつも最終的には問題なく動いている。

ただ、進め方には3つの構造的な失敗があった。

ここでいう「失敗」とは、システムが壊れたという意味ではなく、「気持ちよく、スムーズに開発を進められなかった」という観点の話。

3つの失敗がどうつながったか



定量的には $\text{fix:feat} = 1.03$ 。fixの約半数(145件中約69件、概算)が、実機で動かして初めて発覚するものだった。

失敗1: 環境を分けなかった

実験用の環境と、本番相当の環境を最初から分けなかった。

例えば「機能A」の確認や修正の結果「だけ」を確認したいただけなのに、他のメンバーも弄くってるかもしれない「開発環境」とやらをいちいち触らないといけなかったため、2人以上同じセクションの作業が同時に進められないまま、結果として「実機で初めて動かす瞬間」に負債が集中する構造になってしまった。

また、「作りながら直していこう」という進め方をしていたため、暫定措置のようなものが残り、それが「仮で置いておいたものか」「仕様として決まったものなのか」がかなり曖昧な状態で進んでしまった。

反省点としては「検証用のGCPプロジェクトをメンバー分(少なくともインフラ周りの作業をする人の分)用意しておくべきだった」ということ。

失敗2: レイヤーを絞らなかった

GCPプロジェクトの作成から、Terraformでのインフラ構築、Fast APIでのアプリケーション実装、FreeSwitchやKamailioの設定、運用スクリプトの作成、監視設定、スケールアウトの検証を区切りをつけずにやっていた。

複数同時にやるにしても、せめて「インフラ周り」「アプリ周り」「運用周り」の3つくらいに分けて、1人が同時に触るレイヤーの上限を2つまでに絞るべきだった。

失敗3: エスカレーション基準がなかった

一次情報で裏が取れていない数値を実装に入れる場面や、仕様書に無い値を自分で決める場面が、機械的なルールなしに、その都度の判断で進んでいた。

こうした判断は、後から振り返ると一言確認すればすぐ済んだものが多かった。

判断の余地を残しておく、忙しい時ほど確認が省かれてしまう。

それでも機能した仕組み

- git commit前に敵対的レビューを自動で挟む仕組み
- 実装について、「計画→実装→敵対的検証(別エージェントによるネガティブなレビュー)→修正→再レビュー」のサイクルを回す仕組み
- 教訓をmemoryとして積み上げ、セッションをまたいで引き継ぐ仕組み
- 実機への反映操作(apply・push)は必ず人間が行うという役割分担
- モノレポ構成にしたことで、複数のレイヤーを同時に触る場合でも、コンテキストの保持や変更の追跡が容易になった

次にやること

- 実験用(単体機能の検証用など)・本番相当の環境を、最初から2つ用意する
- 同時に触るレイヤーの上限を2つまでに絞る
- 明らかに判断の余地がある場合、課題管理ツールにチケットを作ってエスカレーションする
 - Slackだと多分流れると思う

まとめ

- 負債の集中は、環境・スコープ・意思決定という進め方の設計に起因する
- 敵対的レビューの自動化・教訓の蓄積・実機操作の人間専管は、AI活用と両立する運用として機能した
- 次の案件では、進め方の設計を着手前に決めておく